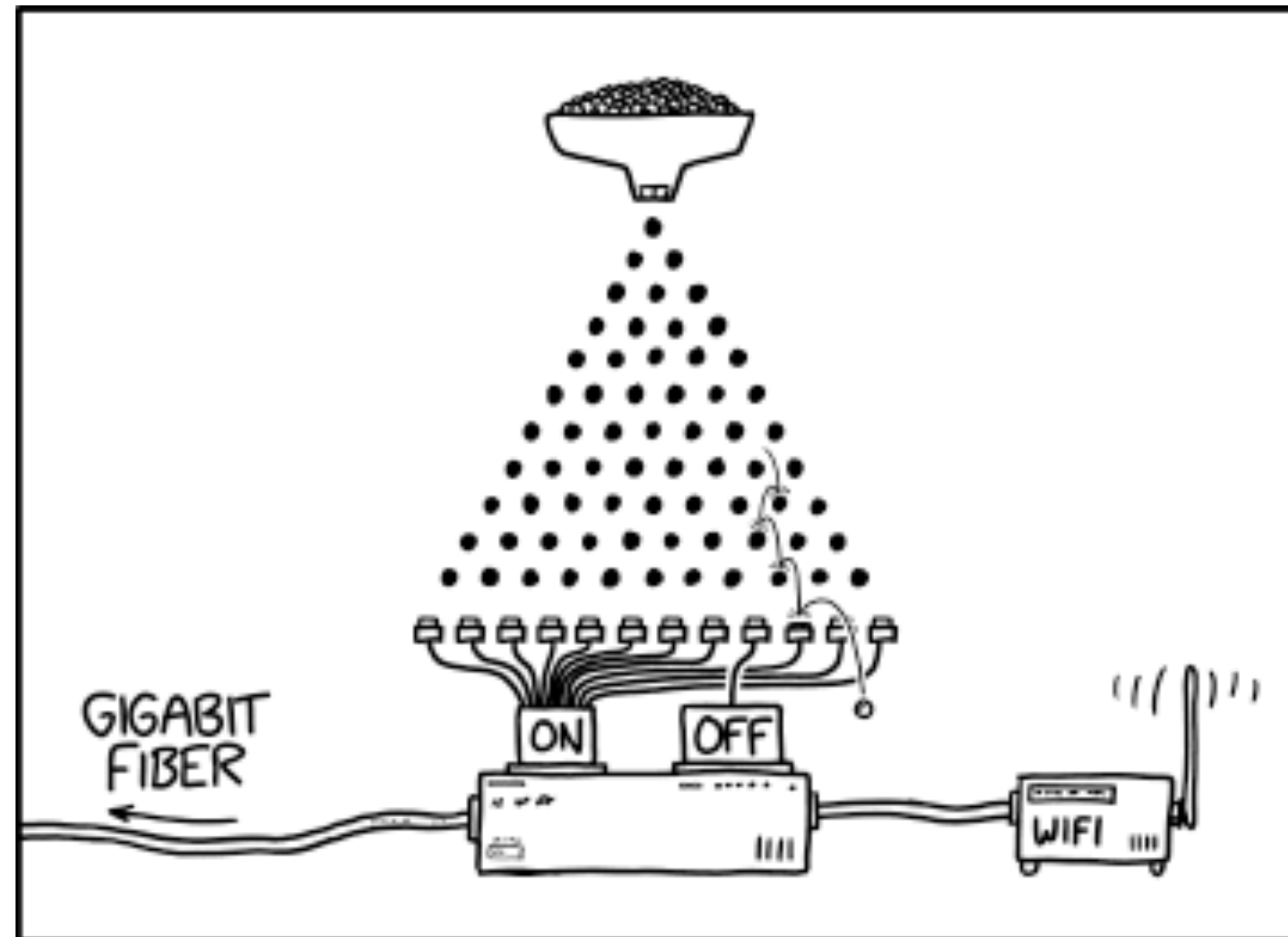


Software Defined Networking



Randall Munroe
<https://xkcd.com/2659/>

MY NEW VACATION SPOT HAS VERY FAST INTERNET THAT TURNS OFF RANDOMLY EVERY NOW AND THEN, JUST SO YOU CAN TELL PEOPLE YOU'LL BE STAYING SOMEWHERE WITHOUT A RELIABLE CONNECTION.

Learning Goals

- ▶ Get an understanding of generalised forwarding
 - ▶ Match-plus-action
 - ▶ Flow Tables
 - ▶ OpenFlow flow tables
- ▶ Get an understanding of software defined networking
 - ▶ Distinction of control and data planes
 - ▶ Benefits of centralisation of the control plane
 - ▶ The SDN stack
 - ▶ OpenFlow protocol

Learning Goal 1FEC82

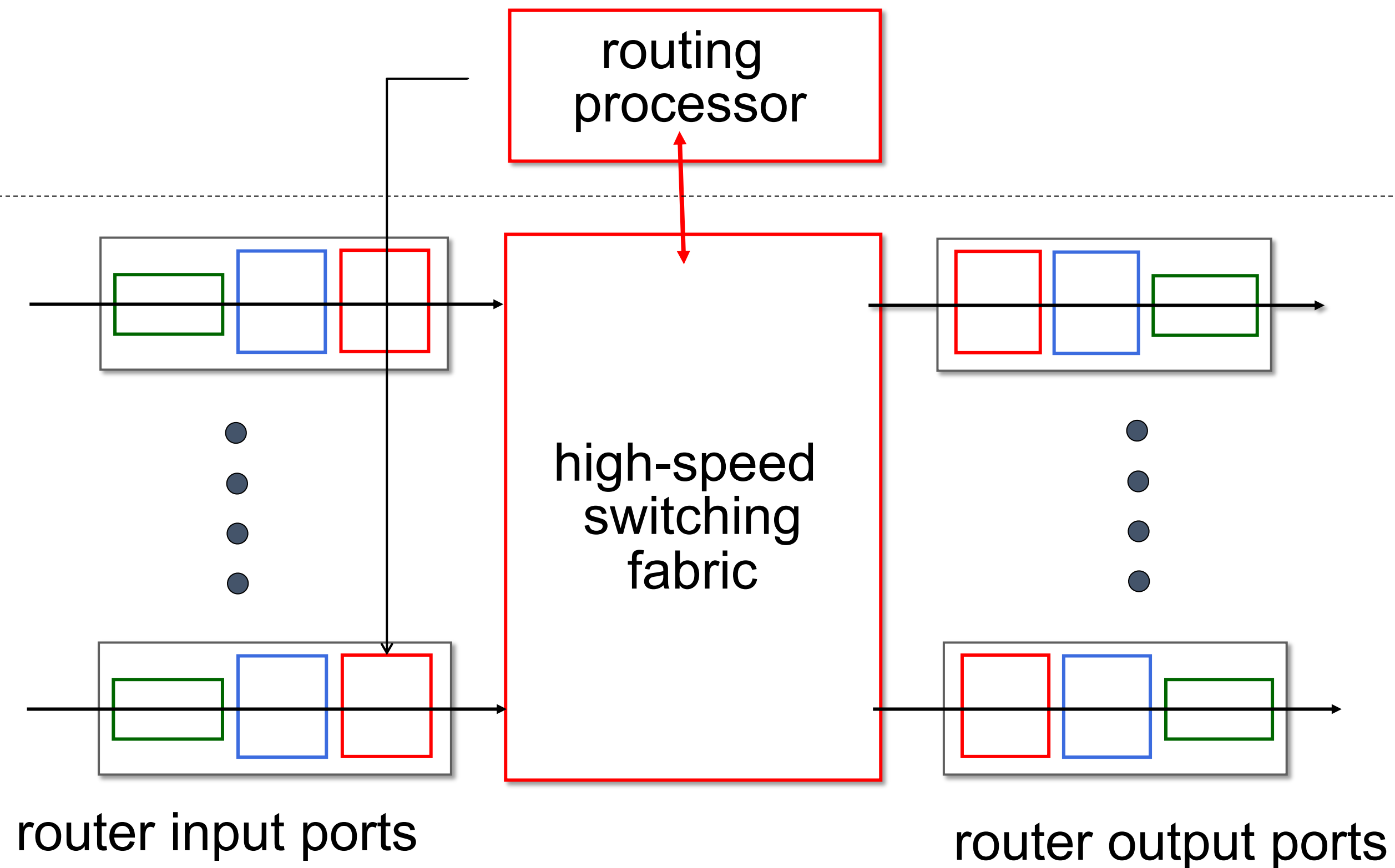
What is generalised forwarding?

Recap: High-level overview of generic router architecture

- ▶ network-wide logic
- ▶ determines how datagrams routed by routers along end-end path from source host to destination host

Control Plane

Data Plane



- ▶ local, per-router function
- ▶ determines how datagram arriving on router input port is forwarded to router output port

Recap: Longest prefix matching

Longest prefix match

When looking for forwarding table entry for given destination address, use *longest* address prefix that matches destination address.

Destination Address Range	Link interface
11001000 00010111 00010*** *****	0
11001000 00010111 00011000 *****	1
11001000 00010111 00011*** *****	2
otherwise	3

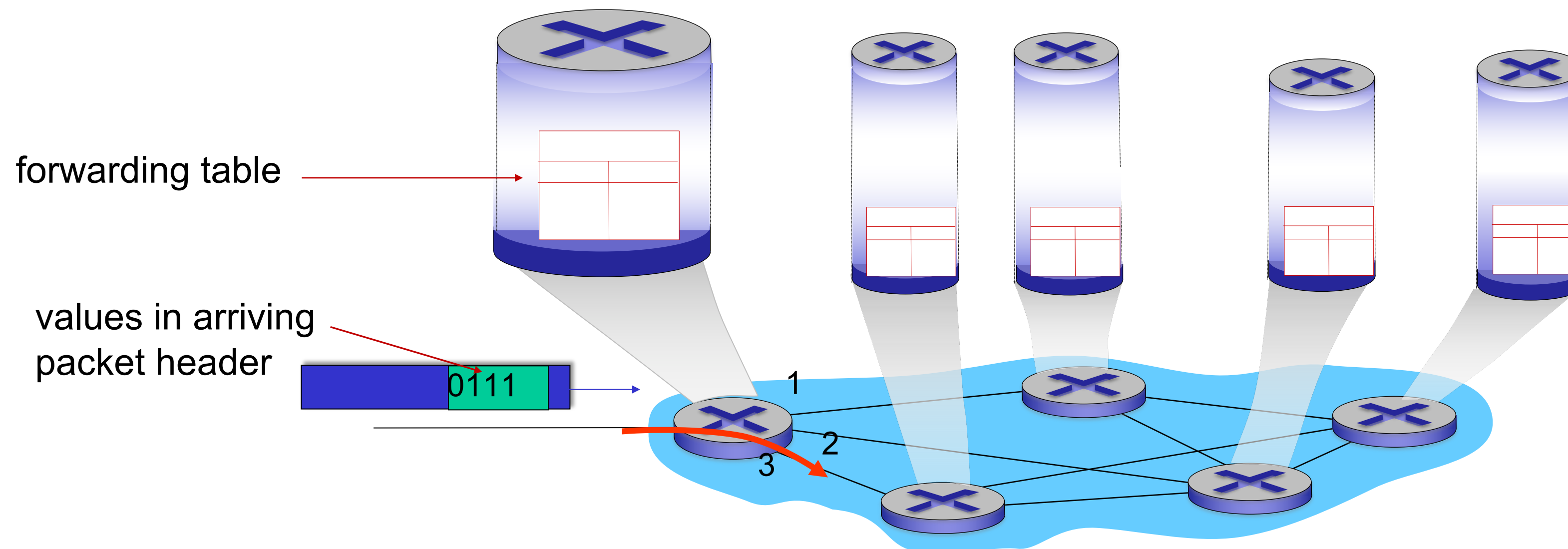
match!

examples:

11001000 00010111 00010110 10100001	which interface?
11001000 00010111 00011000 10101010	which interface?

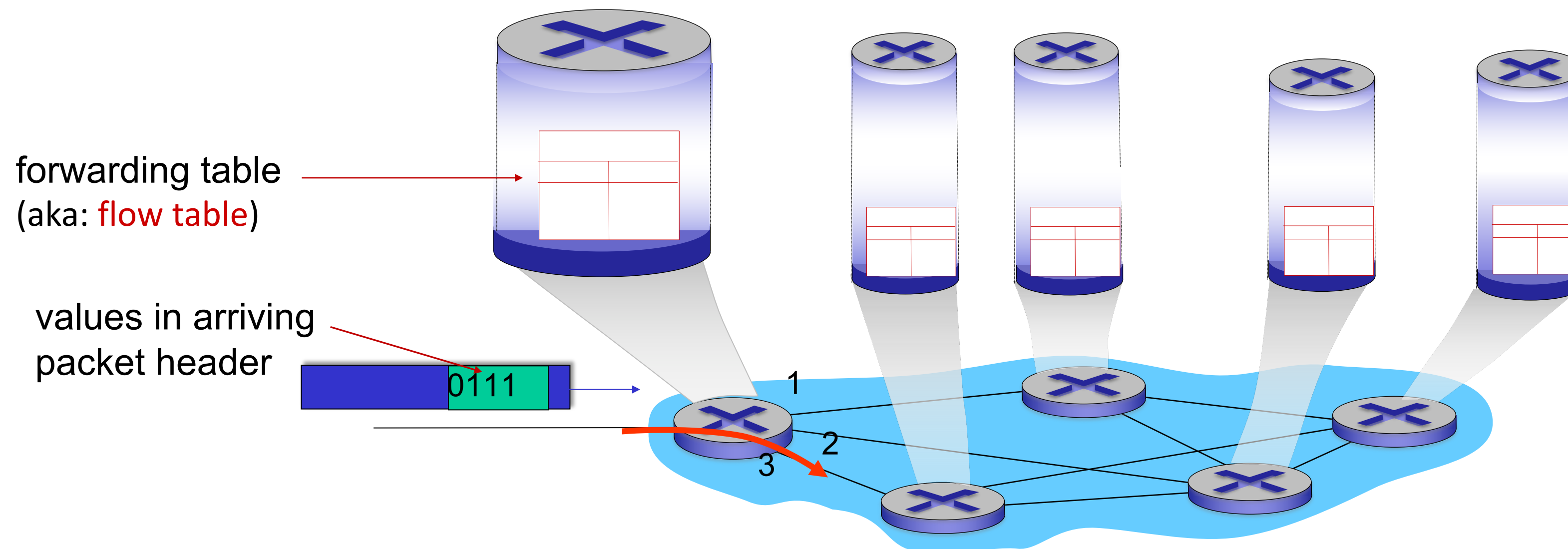
Generalized forwarding: match-plus-action

- ▶ Each router contains a forwarding table
 - ▶ “Match-plus-action” paradigm: match bits in arriving packet & take action
 - ▶ Destination-based forwarding
 - ▶ Match: Destination IP address
 - ▶ Action: Forward



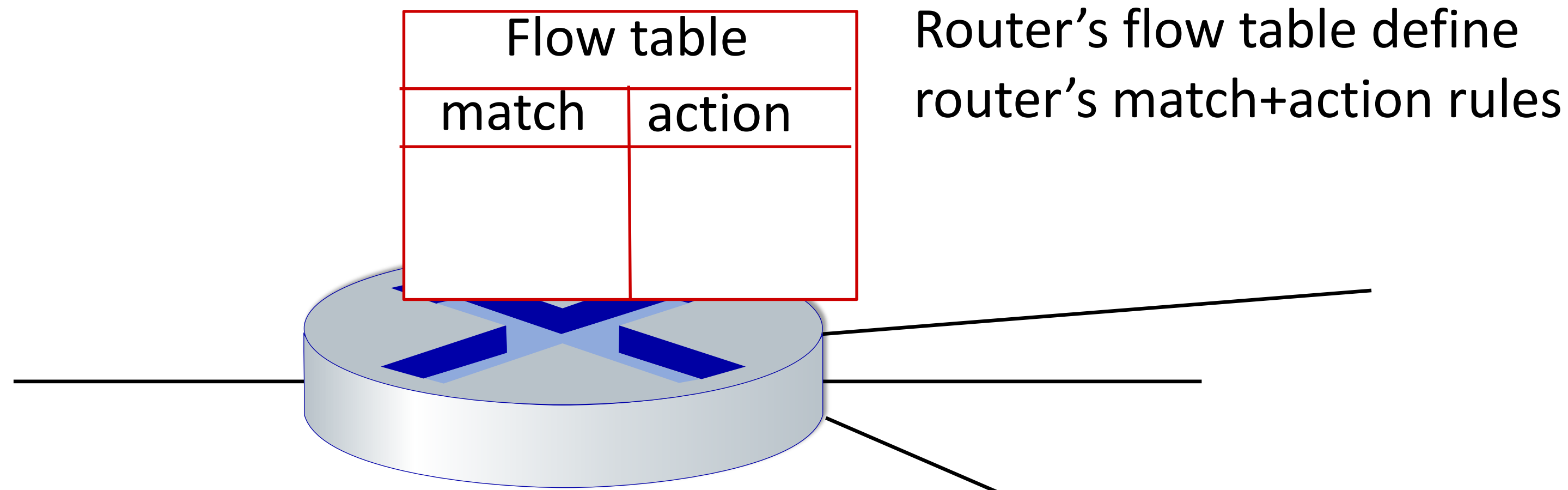
Generalized forwarding: match-plus-action

- ▶ Each router contains a forwarding table (aka: **flow table**)
 - ▶ “Match-plus-action” paradigm: match bits in arriving packet & take action
 - ▶ Destination-based forwarding
 - ▶ Generalised forwarding
 - ▶ Many header fields can determine action
 - ▶ Many actions possible: drop/copy/modify/log packet



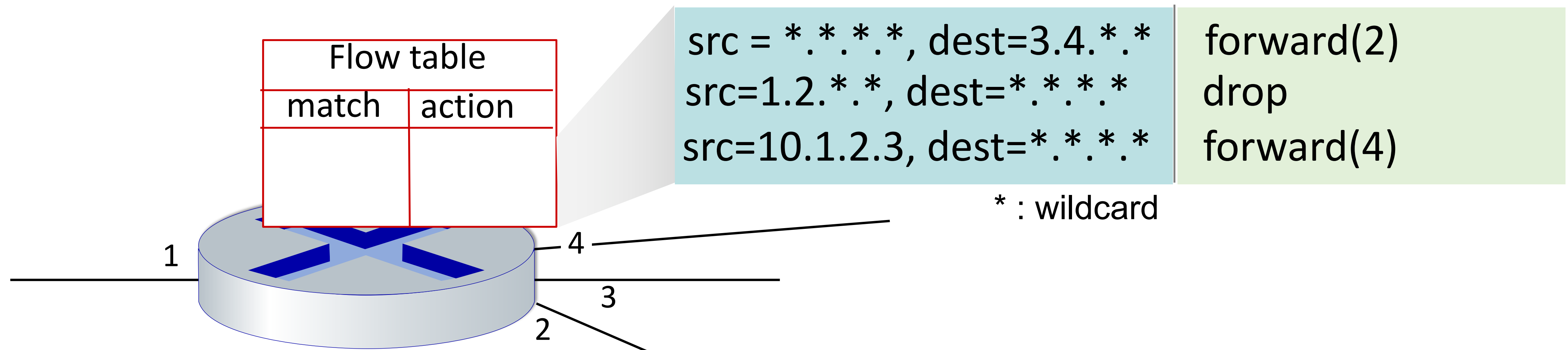
Flow table abstraction

- ▶ **Flow:** defined by header field values (in link-, network-, transport-layer fields)
- ▶ **Generalized forwarding:** simple packet-handling rules
 - ▶ **Match:** pattern values in packet header fields
 - ▶ **Actions:** for matched packet: drop, forward, modify, matched packet or send matched packet to controller
 - ▶ **Priority:** disambiguate overlapping patterns
 - ▶ **Counters:** #bytes and #packets



Flow table abstraction

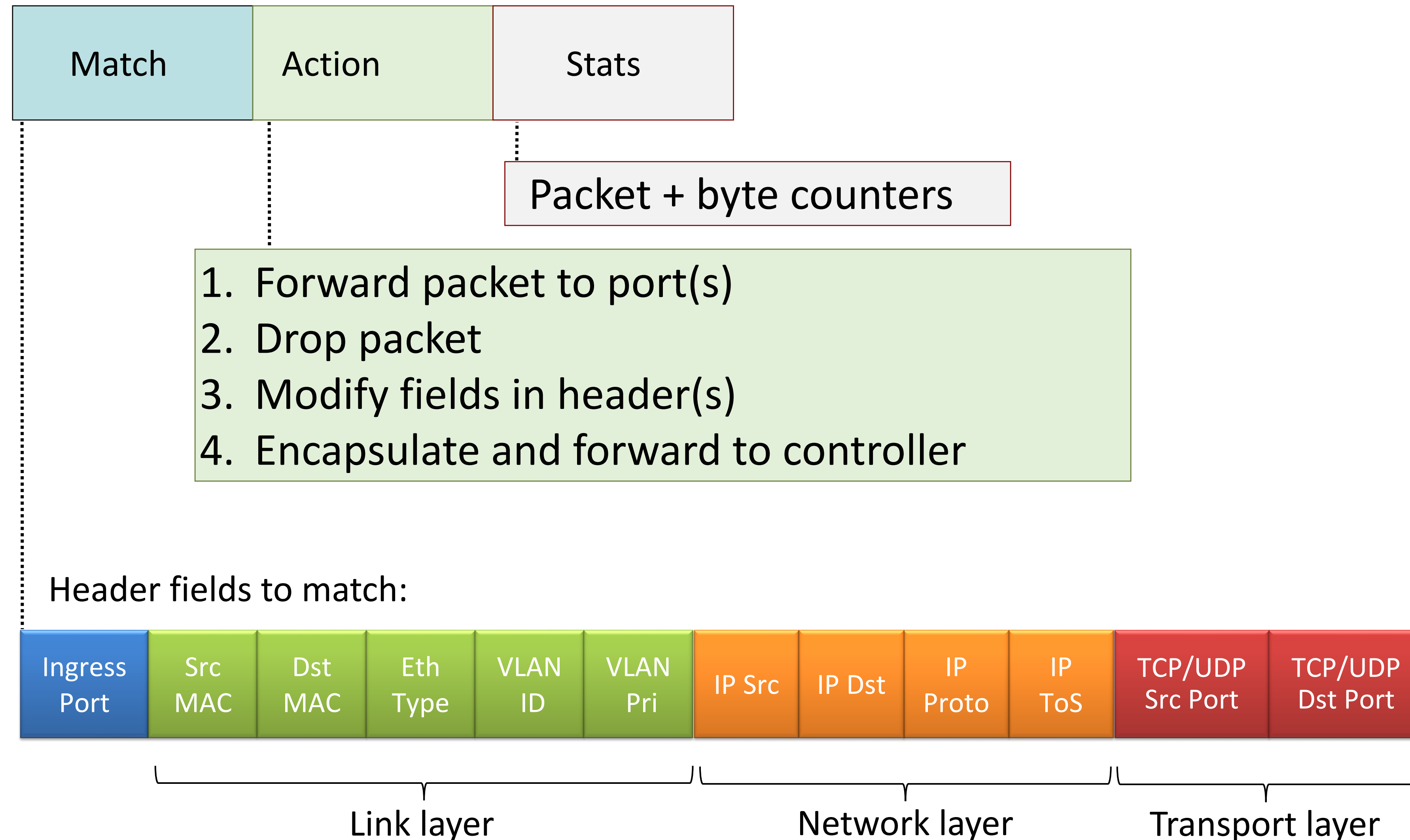
- ▶ **Flow:** defined by header field values (in link-, network-, transport-layer fields)
- ▶ **Generalized forwarding:** simple packet-handling rules
 - ▶ **Match:** pattern values in packet header fields
 - ▶ **Actions:** for matched packet: drop, forward, modify, matched packet or send matched packet to controller
 - ▶ **Priority:** disambiguate overlapping patterns
 - ▶ **Counters:** #bytes and #packets



Flow table abstraction

- ▶ **“Match-plus-action”**: allows to implement a variety of functions
- ▶ **Router**
 - ▶ Match: longest destination IP prefix
 - ▶ Action: forward out of link
- ▶ **Switch**
 - ▶ Match: destination MAC address
 - ▶ Action: forward or flood
- ▶ **NAT**
 - ▶ Match: IP address and port
 - ▶ Action: rewrite address and port
- ▶ **Firewall**
 - ▶ Match: IP addresses and TCP/UDP port numbers
 - ▶ Action: permit or deny

OpenFlow: A standard for flow tables



OpenFlow: Flow table examples

Router – Network layer destination-based forwarding

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	VLAN Pri	IP Src	IP Dst	IP Prot	IP ToS	TCP s-port	TCP d-port	Action
*	*	*	*	*	*	*	51.6.0.8	*	*	*	*	port6

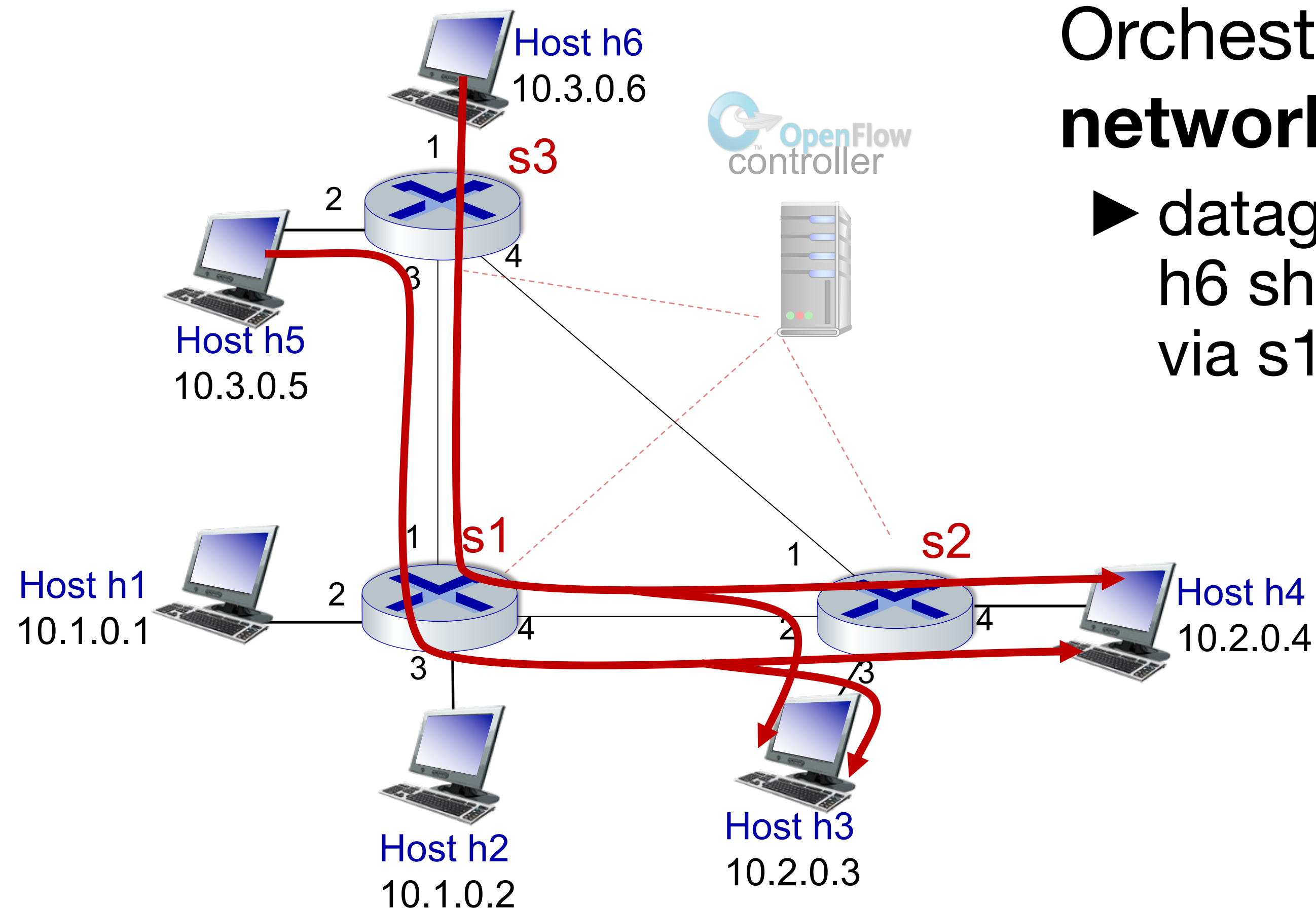
IP datagrams destined to IP address 51.6.0.8 should be forwarded to router output port 6

Switch – Link Layer destination-based forwarding

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	VLAN Pri	IP Src	IP Dst	IP Prot	IP ToS	TCP s-port	TCP d-port	Action
*	*	22:A7:23:11:E1:02	*	*	*	*	*	*	*	*	*	port3

layer 2 frames with destination MAC address 22:A7:23:11:E1:02 should be forwarded to output port 3

OpenFlow Example

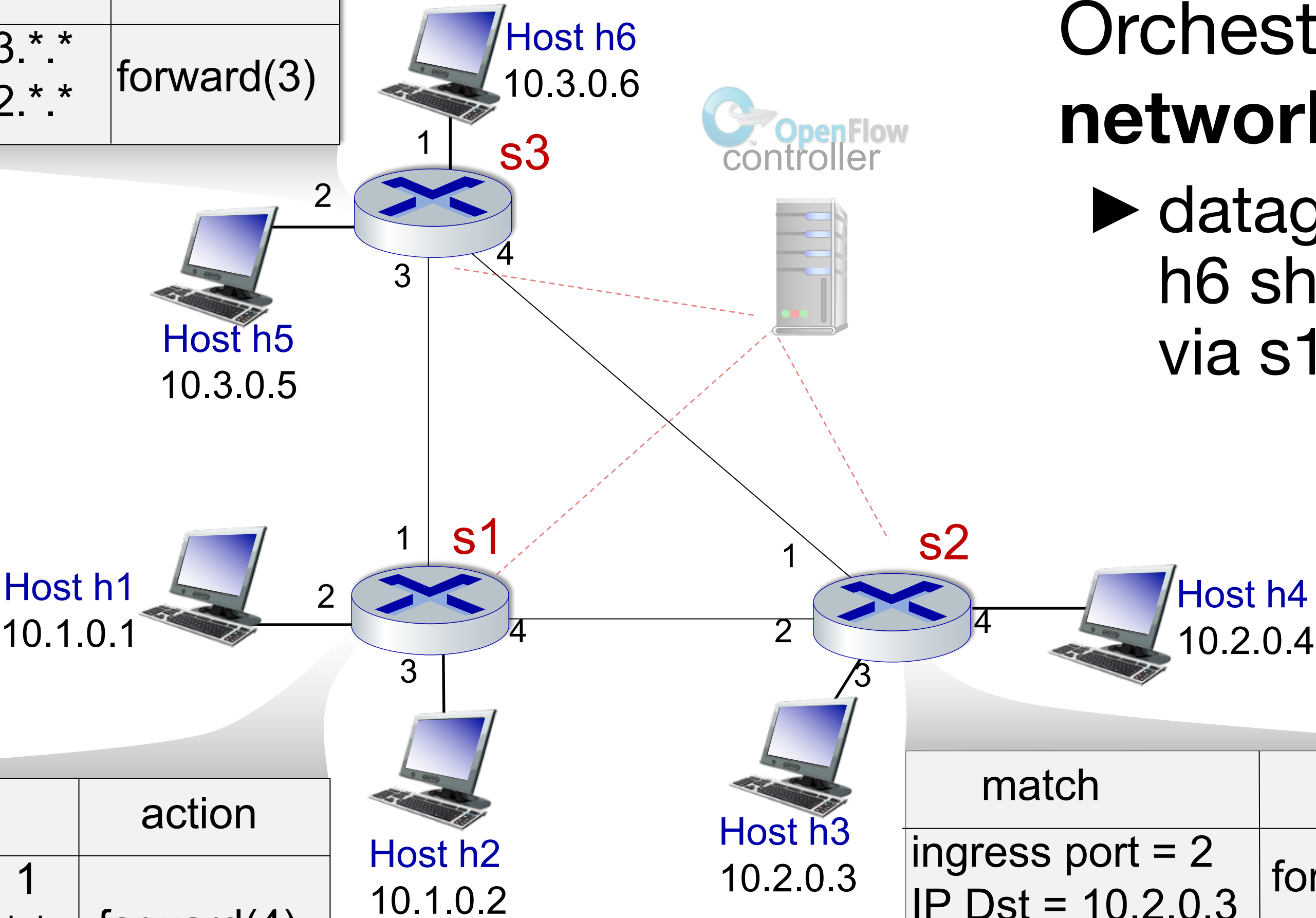


Orchestrated tables can create **network-wide** behavior, e.g.,:

- datagrams from hosts h5 and h6 should be sent to h3 or h4, via s1 and from there to s2

OpenFlow Example

match	action
IP Src = 10.3.*.* IP Dst = 10.2.*.*	forward(3)



match	action
ingress port = 1 IP Src = 10.3.*.* IP Dst = 10.2.*.*	forward(4)

Orchestrated tables can create **network-wide** behavior, e.g.,:

- datagrams from hosts h5 and h6 should be sent to h3 or h4, via s1 and from there to s2

match	action
ingress port = 2 IP Dst = 10.2.0.3	forward(3)
ingress port = 2 IP Dst = 10.2.0.4	forward(4)

Generalized forwarding Summary

- ▶ “Match-plus-action”: match bits in arriving packet header(s) in any layers, take action
 - ▶ Matching over many fields (link-, network-, transport-layer)
 - ▶ Local actions: drop, forward, modify, or send matched packet to controller
 - ▶ “Program” network-wide behaviours
- ▶ Simple form of “network programmability”
 - ▶ Programmable, per-packet “processing”
 - ▶ Historical roots: active networking
 - ▶ Today: more generalized programming
 - ▶ P4 (see p4.org).

Learning Goal 4A6B25

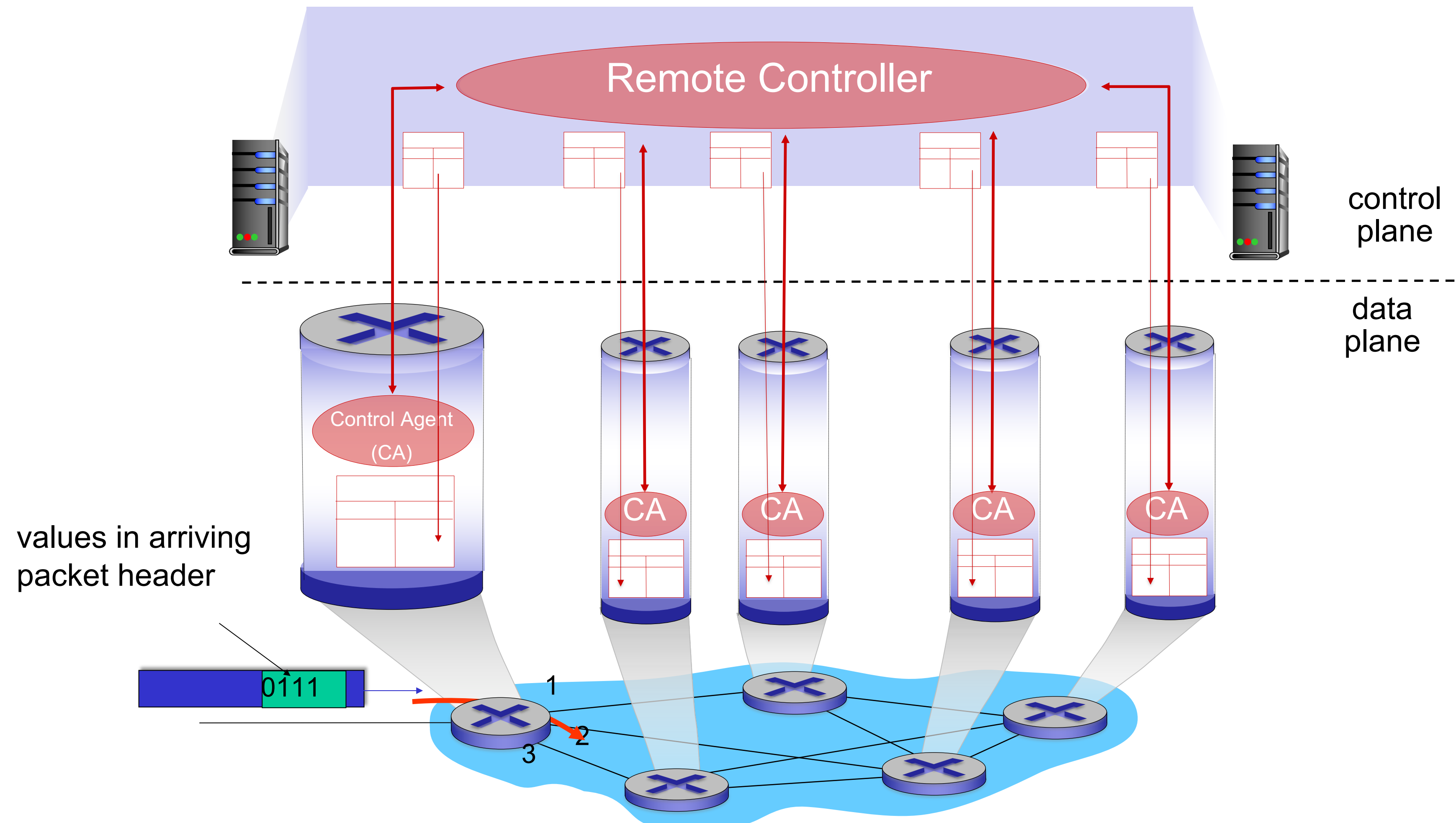
What is Software Defined Networking?

Software Defined Networking (SDN)

- ▶ Network layer: historically implemented via distributed, per-router control approach
 - ▶ Monolithic router contains switching hardware, runs proprietary implementation of Internet standard protocols in proprietary router OS
 - ▶ Different “middleboxes” for different network layer functions: firewalls, load balancers, NAT boxes, ..
- ▶ ~2005: Renewed interest in rethinking network control plane

Software-Defined Networking (SDN) control plane

- Remote controller computes, installs forwarding tables in routers

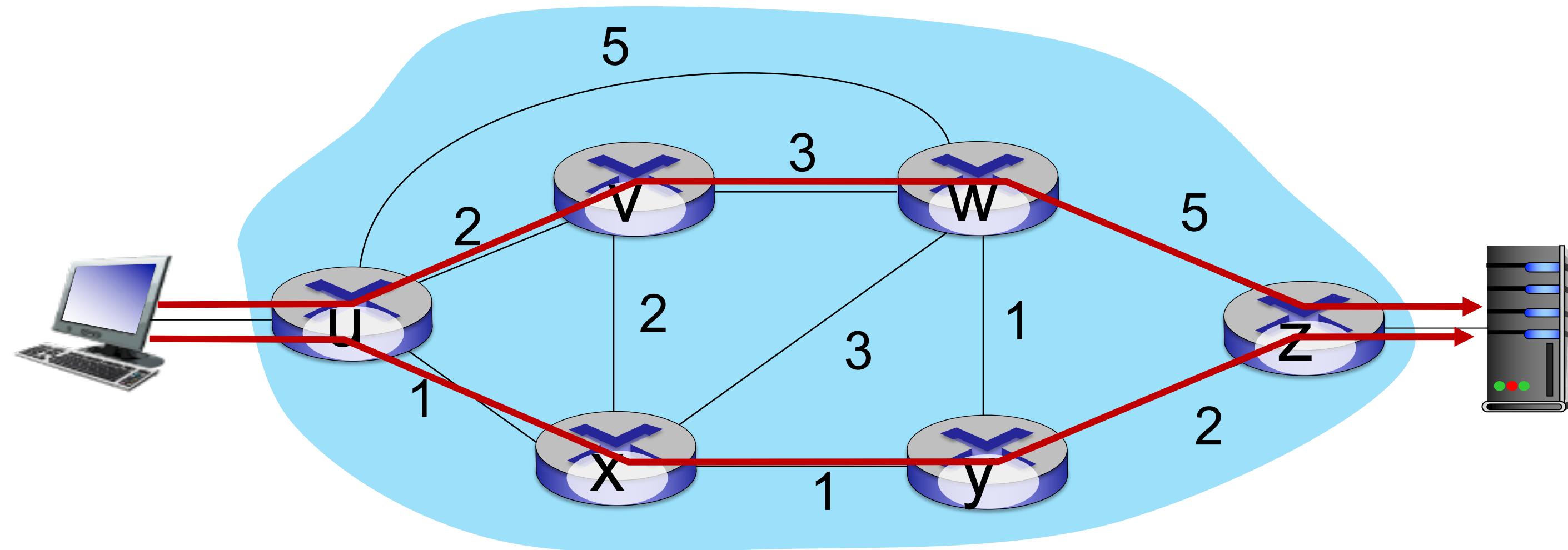


Adapted from slides by J.F Kurose and K.W. Ross

Centralisation of the control plane

- ▶ Why a logically centralized control plane?
 - ▶ **Easier network management:** Avoid router misconfigurations, greater flexibility of traffic flows
 - ▶ **Table-based forwarding:** Allows “programming” routers (recall OpenFlow API)
 - ▶ Centralized “programming” easier: Compute tables centrally and distribute
 - ▶ Distributed “programming” more difficult: Compute tables as result of distributed algorithm (protocol) implemented in each-and-every router
 - ▶ **Open:** Non-proprietary implementation of control plane
 - ▶ Foster innovation

Traffic engineering is difficult with traditional routing

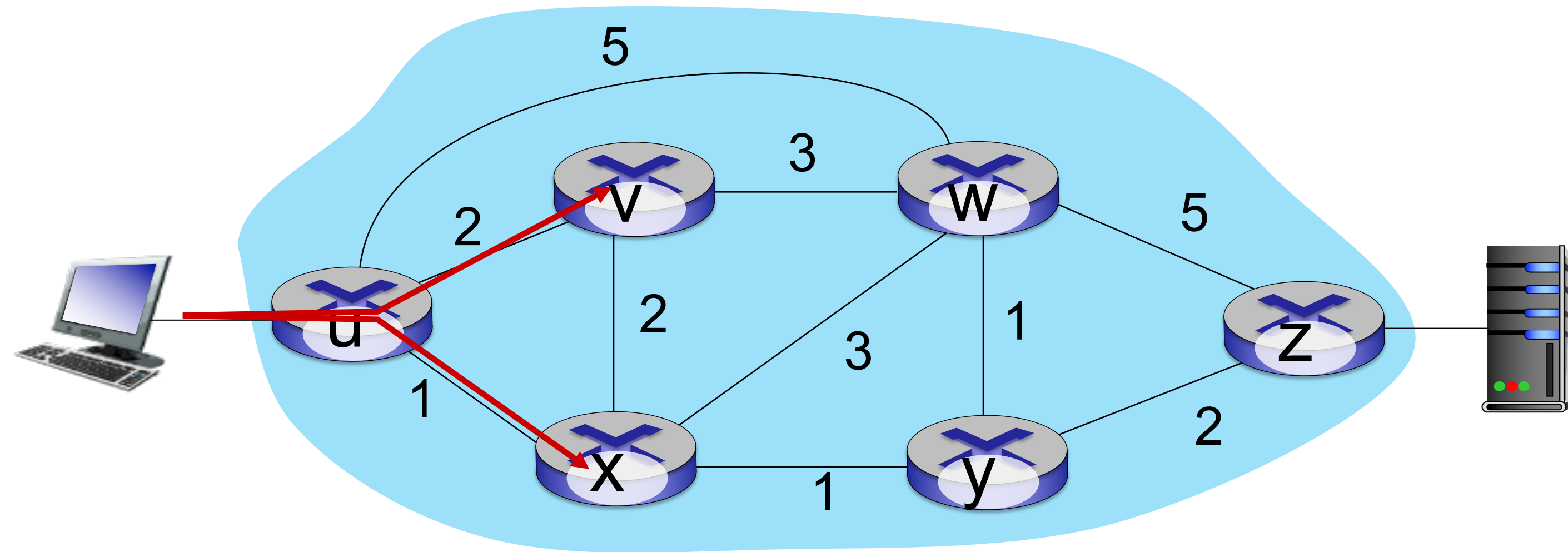


Q: What if network operator wants u-to-z traffic to flow along uvwz, rather than uxyz?

A: Need to re-define link weights so traffic routing algorithm computes routes accordingly (or need a new routing algorithm)!

Link weights are only control “knobs”: not much control!

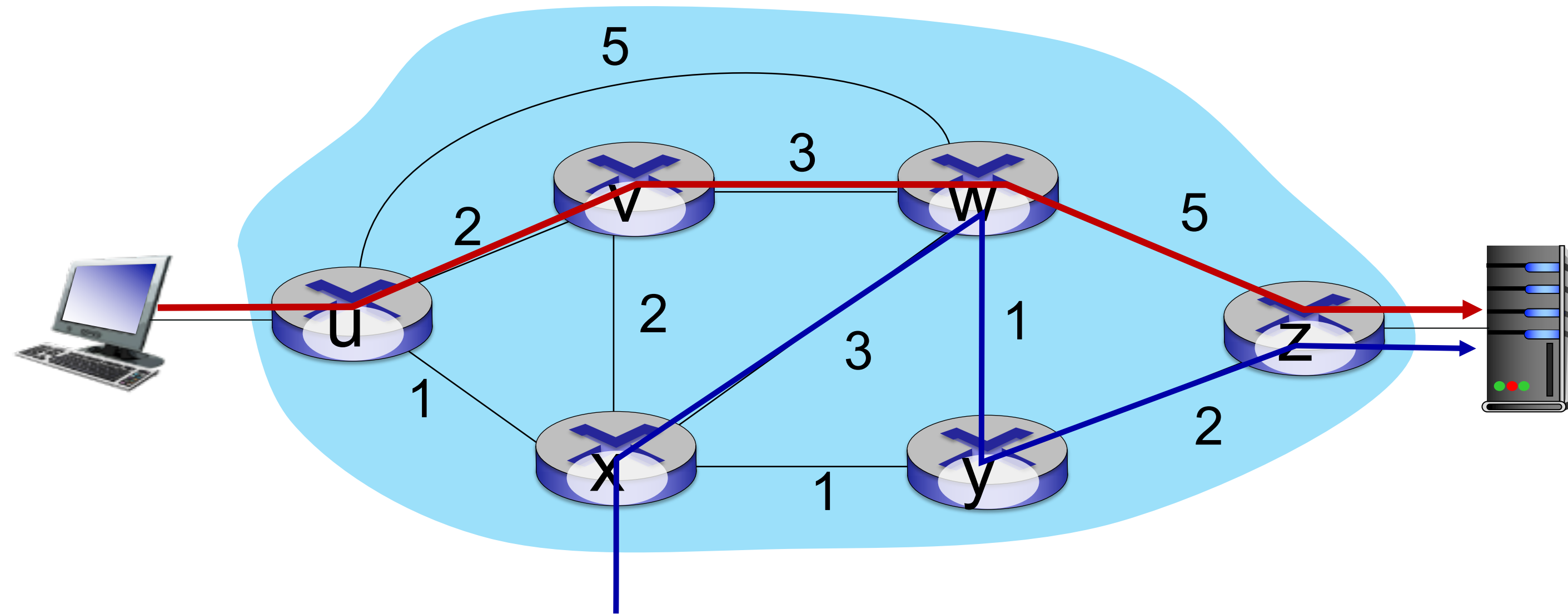
Traffic engineering is difficult with traditional routing



Q: What if network operator wants to split u-to-z traffic along uvwz and uxyz (load balancing)?

A: Can't do it (or need a new routing algorithm)

Traffic engineering is difficult with traditional routing

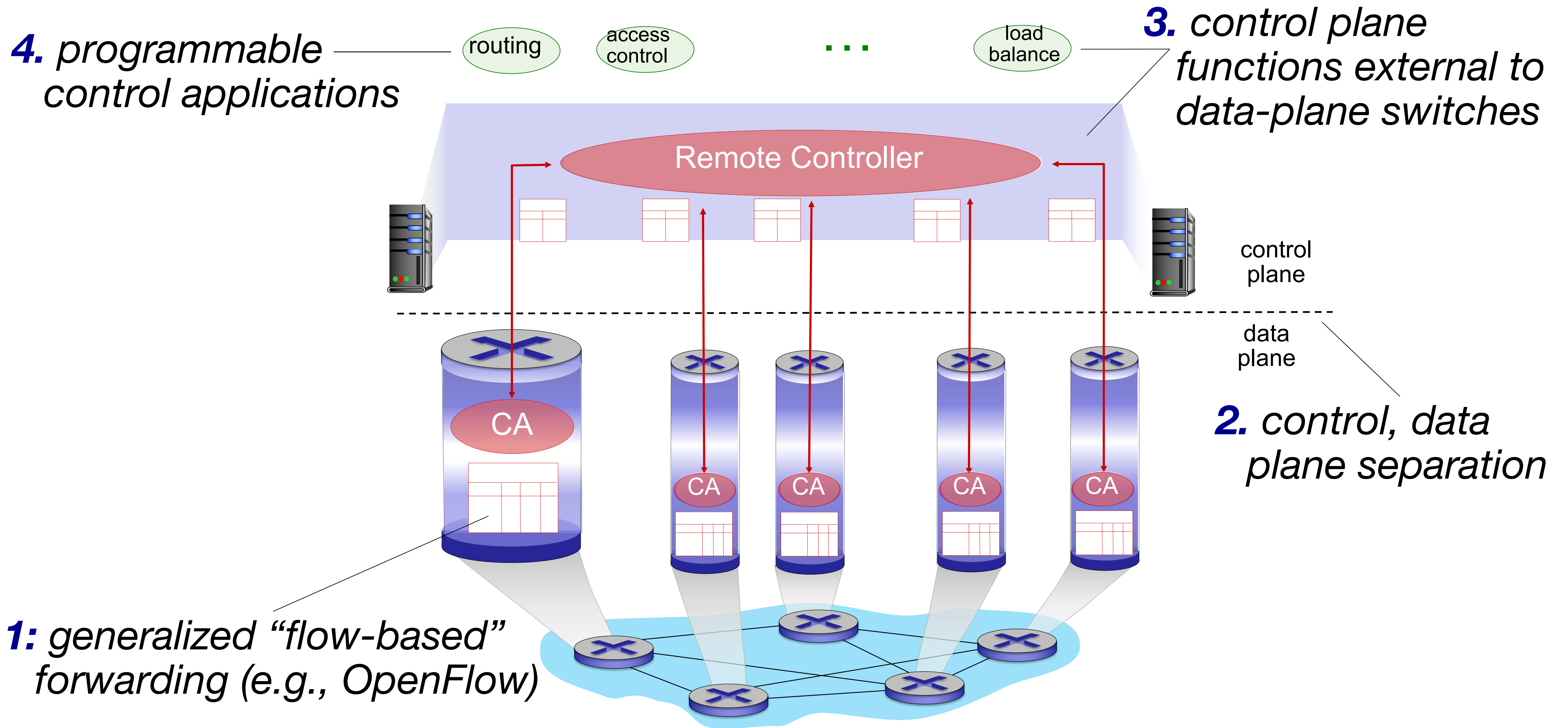


Q: What if w wants to route blue and red traffic differently from w to z?

A: Can't do it (with destination-based forwarding and traditional routing algorithms)

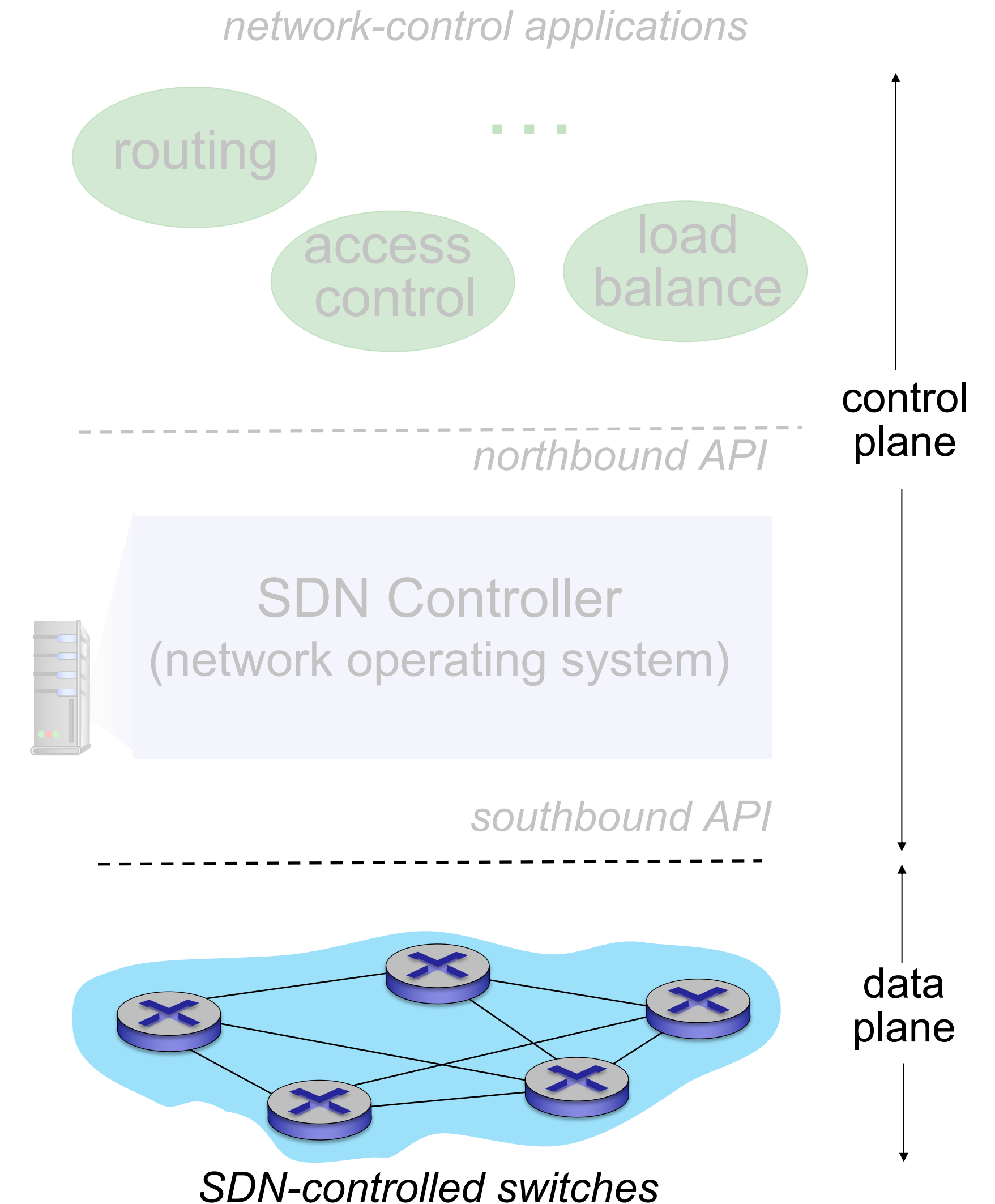
Remember: Generalized forwarding (and thus SDN) can be used to achieve *any* routing desired

Software defined networking (SDN) conceptual overview



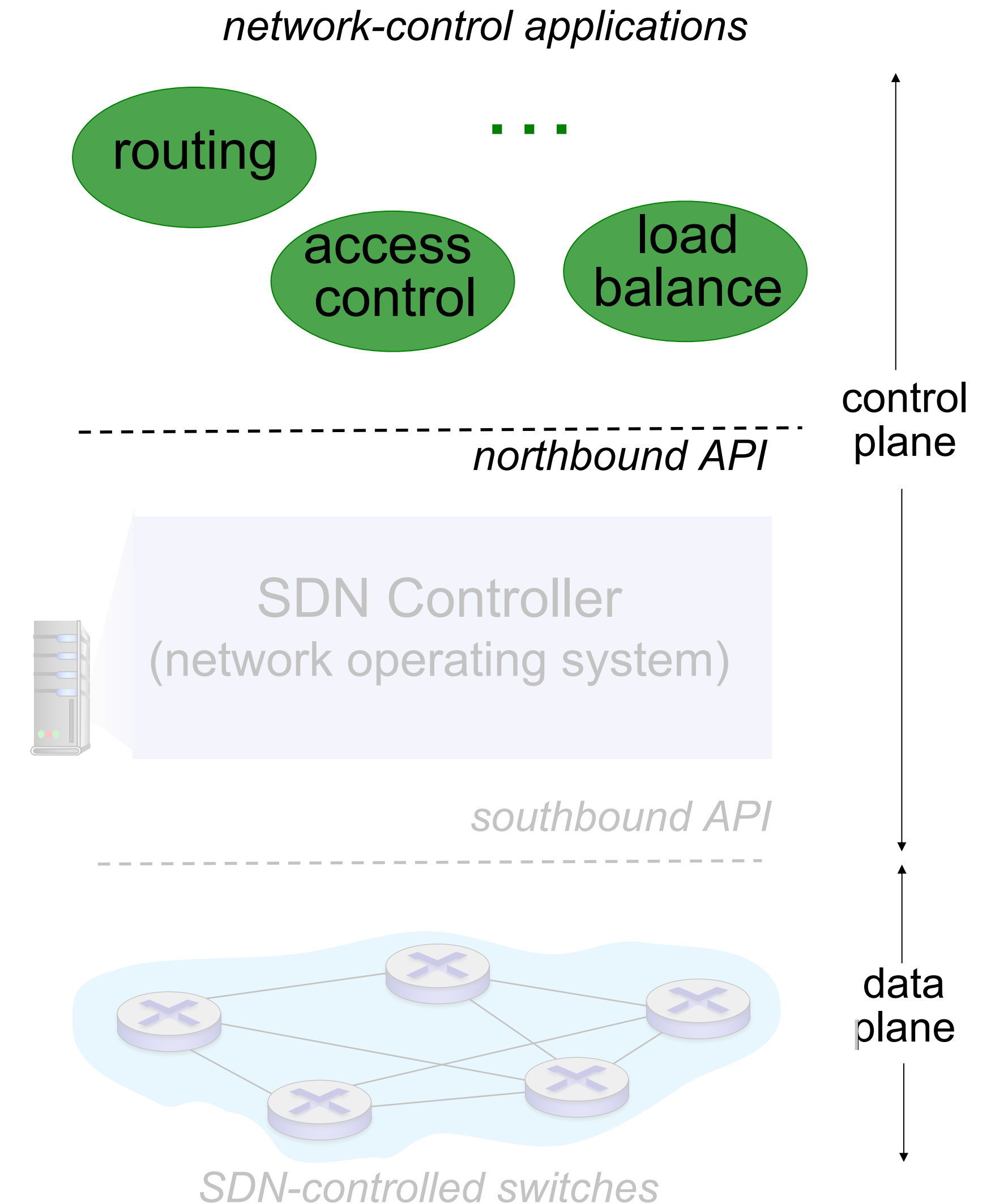
The software defined networking (SDN) stack

- ▶ Data-plane switches
 - ▶ Fast, simple, commodity switches implementing generalized data-plane forwarding in hardware
 - ▶ Flow (forwarding) table computed, installed under controller supervision
 - ▶ Protocol for communicating with controller and API for table-based switch control (e.g., OpenFlow)
 - ▶ Defines what is controllable, what is not



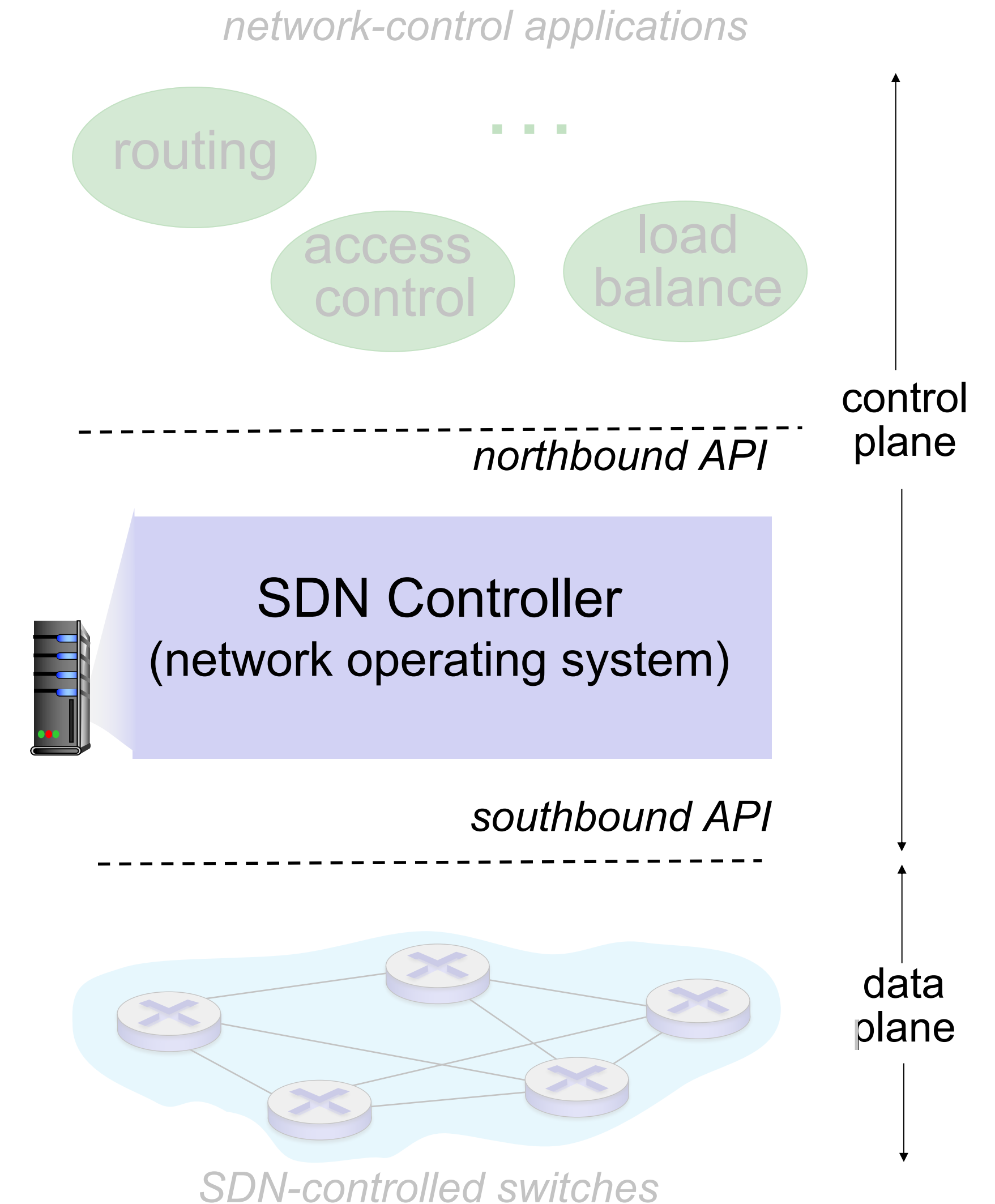
The software defined networking (SDN) stack

- ▶ Network-control apps
 - ▶ “Brains” of control: implement control functions using lower-level services, API provided by SDN controller
 - ▶ Unbundled: can be provided by 3rd party
 - ▶ Distinct from routing vendor or SDN controller



The software defined networking (SDN) stack

- ▶ SDN controller
 - ▶ Maintains network state information
 - ▶ Interacts with network control applications “above” via northbound API
 - ▶ Interacts with network switches “below” via southbound API
 - ▶ Implemented as distributed system for performance, scalability, fault-tolerance, robustness

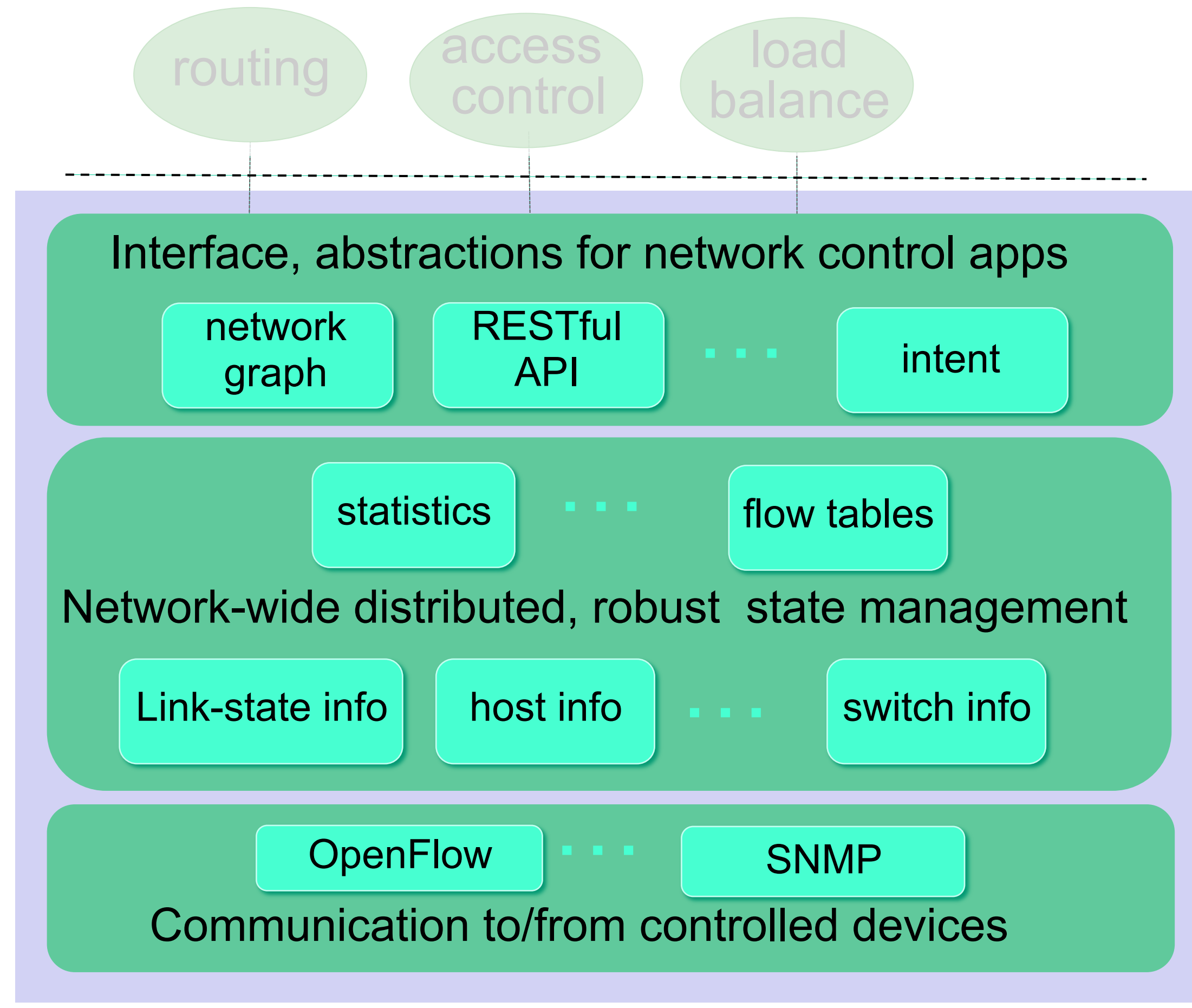


Components of SDN controller

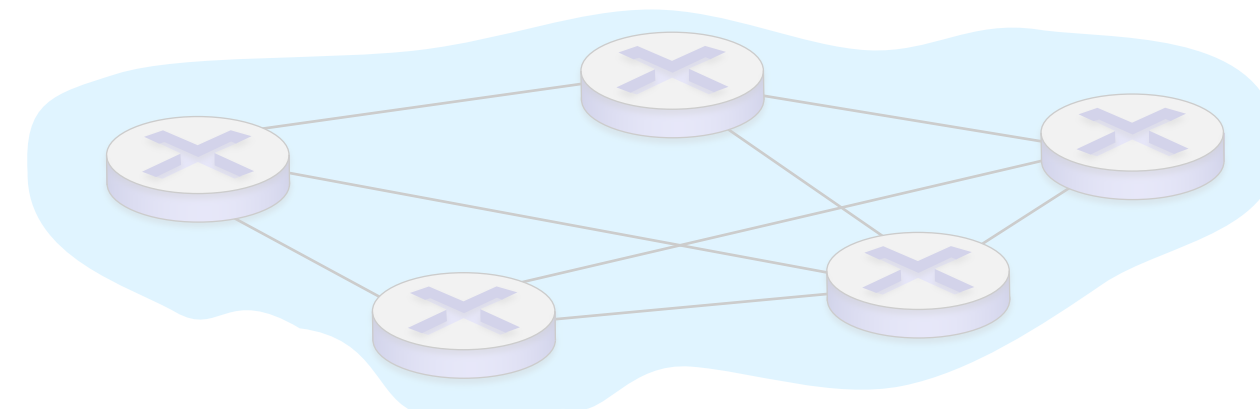
interface layer to network control apps: abstractions API

network-wide state management: state of networks links, switches, services, a distributed database

communication: communicate between SDN controller and controlled switches



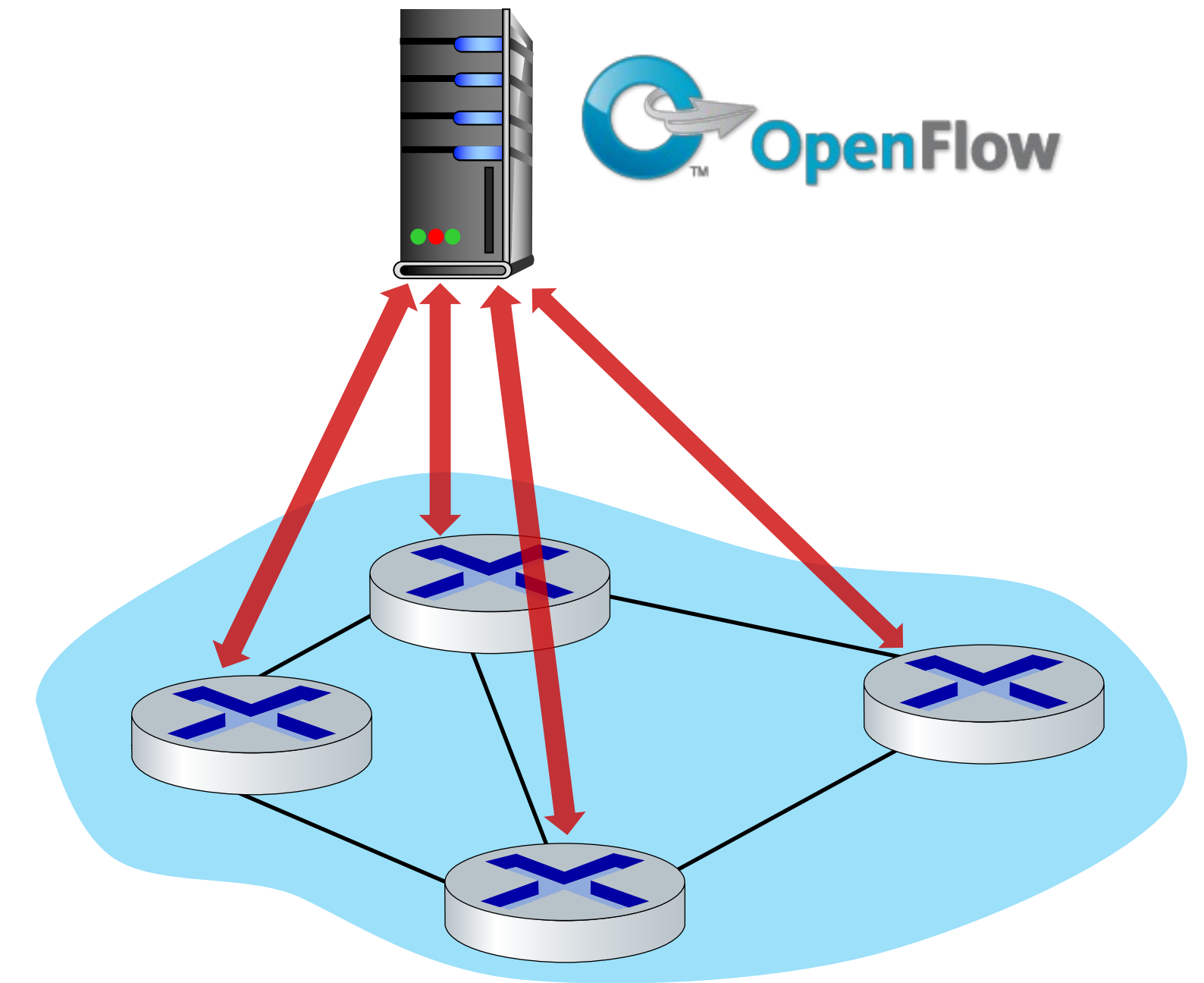
SDN
controller



OpenFlow protocol

- ▶ Operates between controller and switch
- ▶ TCP used to exchange messages
 - ▶ Optional encryption
- ▶ Messages can be sent in both ways
 - ▶ Controller-to-switch
 - ▶ Switch-to-controller

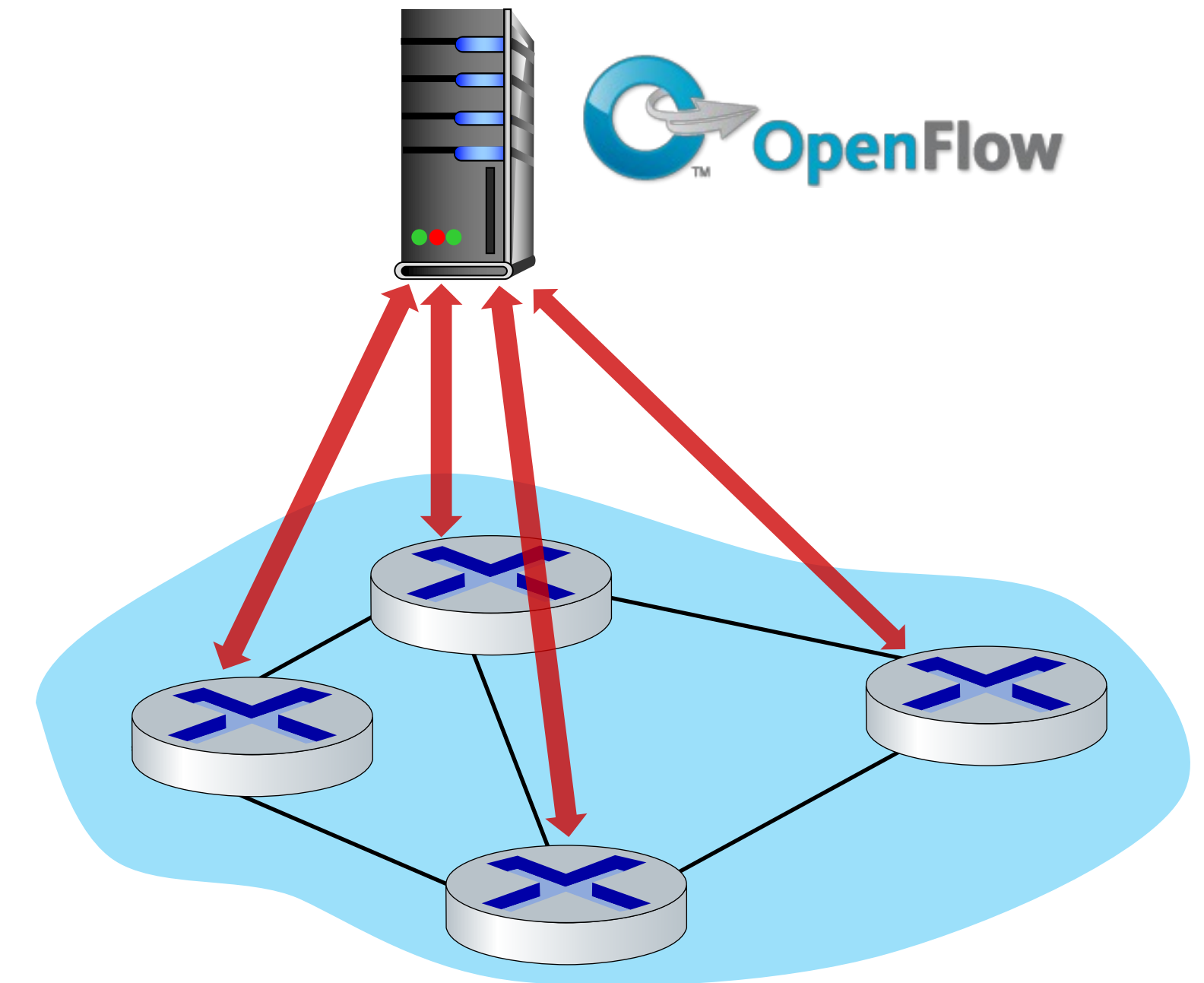
OpenFlow Controller



OpenFlow: Controller-to-switch messages

- ▶ Key controller-to-switch messages
 - ▶ **Configuration:** controller queries/sets switch configuration parameters
 - ▶ **Modify-State:** add, delete, modify flow entries in the OpenFlow tables
 - ▶ **Read-state:** collect statistics and counter values
 - ▶ **Send-packet:** controller can send this packet out of specific switch port

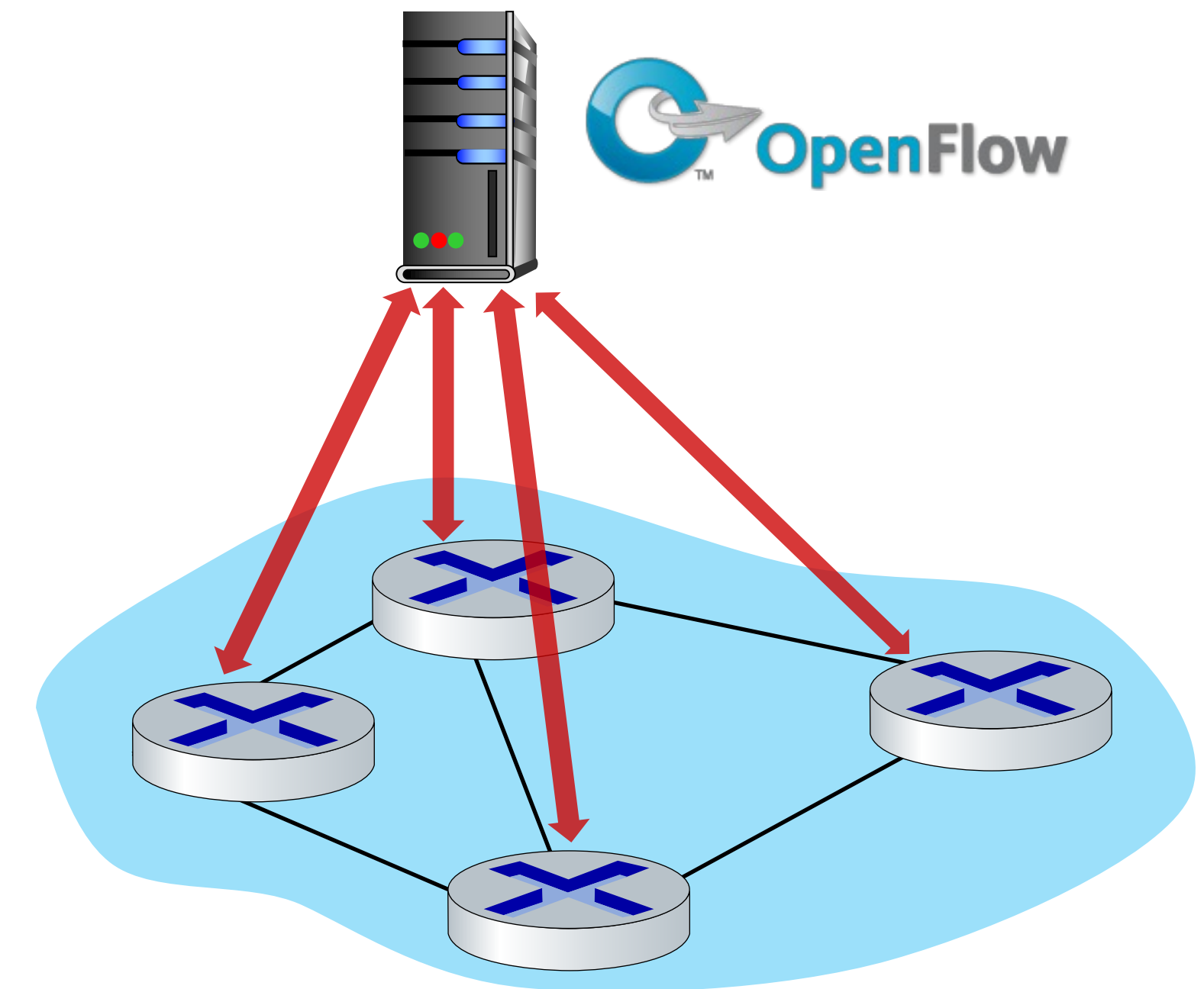
OpenFlow Controller



OpenFlow: Switch-to-controller messages

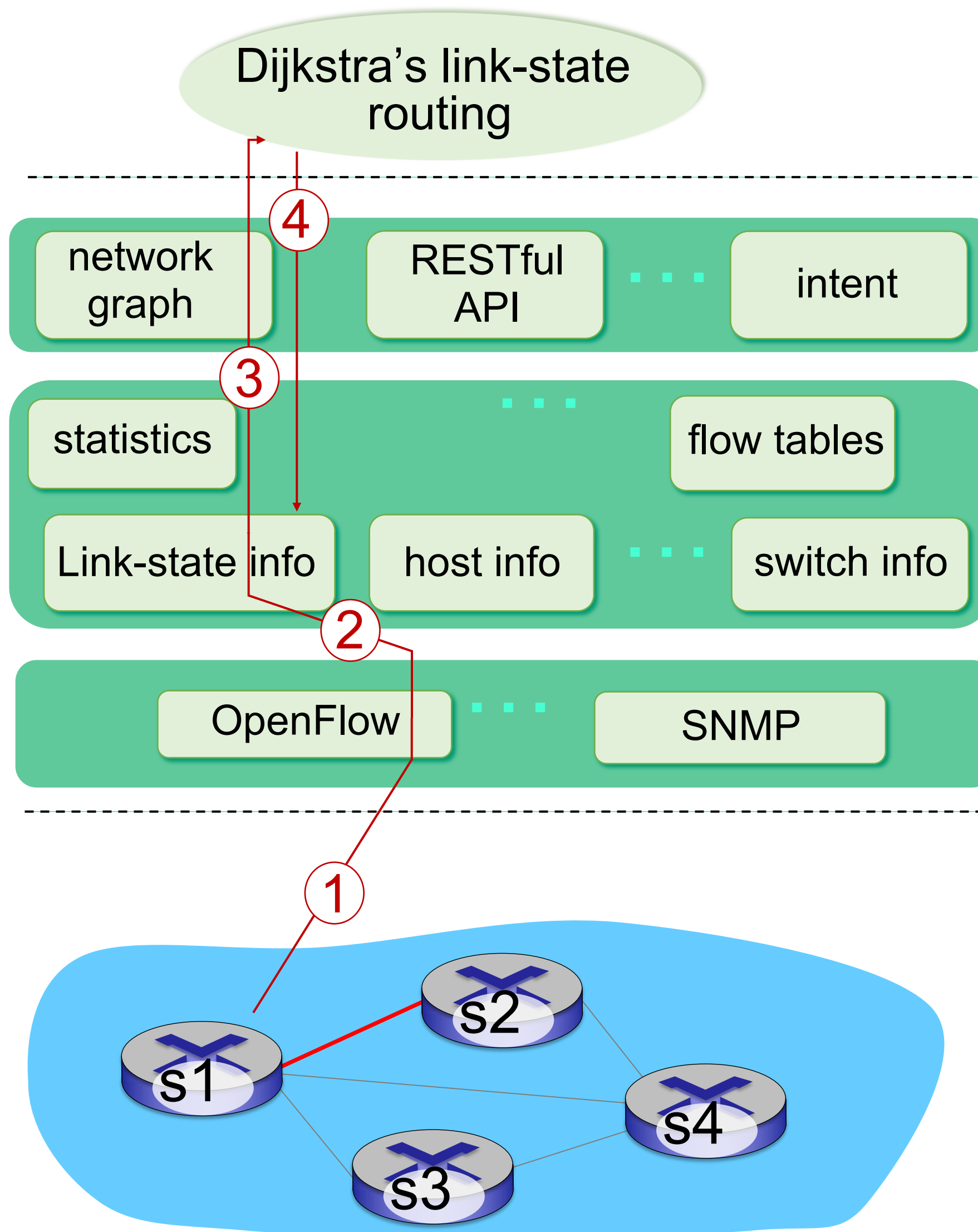
- ▶ Key switch-to-controller messages
 - ▶ **Packet-In**: transfer packet (and its control) to controller
 - ▶ **Flow-Removed**: flow table entry deleted at switch
 - ▶ **Port-Status**: inform controller of a change on a port.

OpenFlow Controller



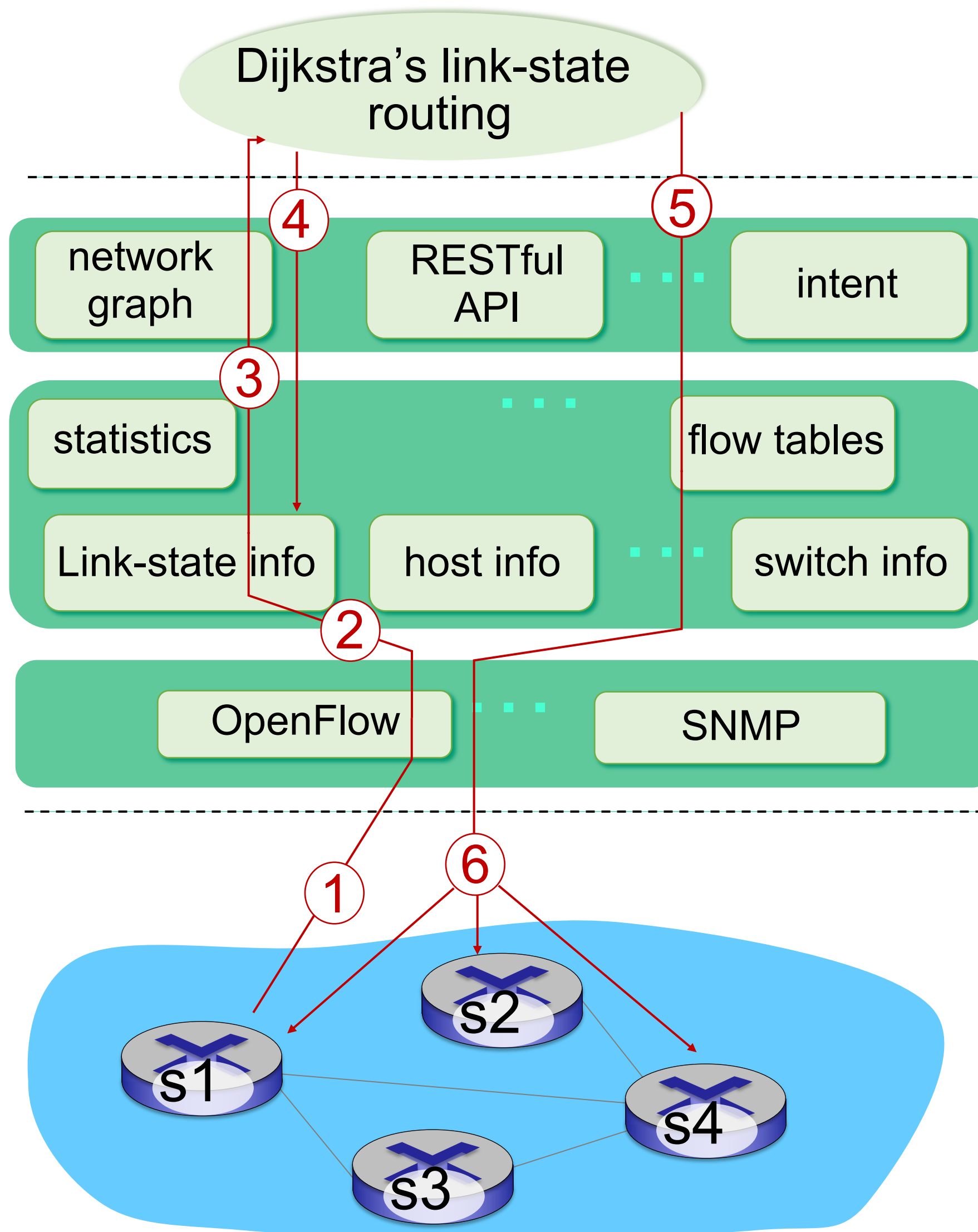
Fortunately, network operators don't "program" switches by creating/sending OpenFlow messages directly → Instead use higher-level abstraction at controller

SDN: Control/data plane interaction example



- ① S1, experiencing link failure uses OpenFlow port status message to notify controller
- ② SDN controller receives OpenFlow message, updates link status info
- ③ Dijkstra's routing algorithm application has previously registered to be called when ever link status changes. It is called.
- ④ Dijkstra's routing algorithm accesses network graph info, link state info in controller, computes new routes

SDN: Control/data plane interaction example



- ⑤ link state routing app interacts with flow-table-computation component in SDN controller, which computes new flow tables needed
- ⑥ controller uses OpenFlow to install new tables in switches that need updating

SDN and the future of traditional network protocols

- ▶ SDN-computed versus router-computer forwarding tables
 - ▶ Just one example of logically-centralized-computed versus protocol computed
- ▶ One could imagine SDN-computed congestion control
 - ▶ Controller sets sender rates based on router-reported (to controller) congestion levels

Learning Goals

- ▶ Get an understanding of generalised forwarding ✓
 - ▶ Match-plus-action
 - ▶ Flow Tables
 - ▶ OpenFlow flow tables
- ▶ Get an understanding of software defined networking ✓
 - ▶ Distinction of control and data planes
 - ▶ Benefits of centralisation of the control plane
 - ▶ The SDN stack
 - ▶ OpenFlow protocol